



Optimizing Code for Better Cache Performance

CMSC 14400

Prof. Yanjing Li

Today

- **How to write code that is cache friendly?**
- **Rearranging loops to improve spatial locality**
- **Using blocking to improve temporal locality**

Review: Locality Example

- **Question:** Can you make this function more cache friendly?
 - Yes
- **Maximize spatial locality by accessing elements that are contiguous in memory as much as possible**
 - Use the same principle to access any data structure (e.g., any dimensional array of any type, structs, etc.)

```
int sum_array_3d(int a[N][N][N])
{
    int i, j, k, sum = 0;

    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            for (k = 0; k < N; k++)
                sum += a[k][i][j];

    return sum;
}
```

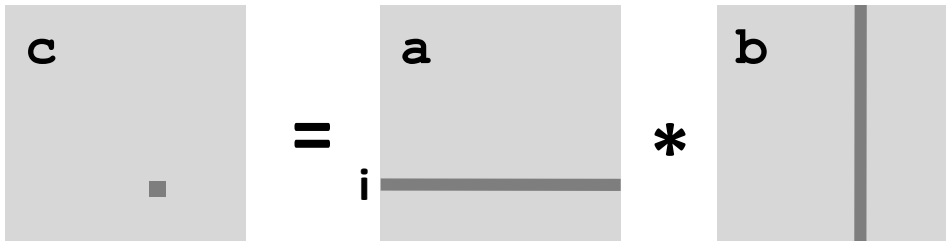
these are
jumps in
memory and
∴ slower

Today

- **How to write code that is cache friendly?**
- **Rearranging loops to improve spatial locality**
- **Using blocking to improve temporal locality**

Example: Matrix Multiplication

```
c = (double *) calloc(sizeof(double), n*n);  
  
/* Multiply n x n matrices a and b */  
void mmm(double *a, double *b, double *c, int n) {  
    int i, j, k;  
    for (i = 0; i < n; i++)  
        for (j = 0; j < n; j++)  
            for (k = 0; k < n; k++)  
                c[i*n + j] += a[i*n + k] * b[k*n + j];  
}
```



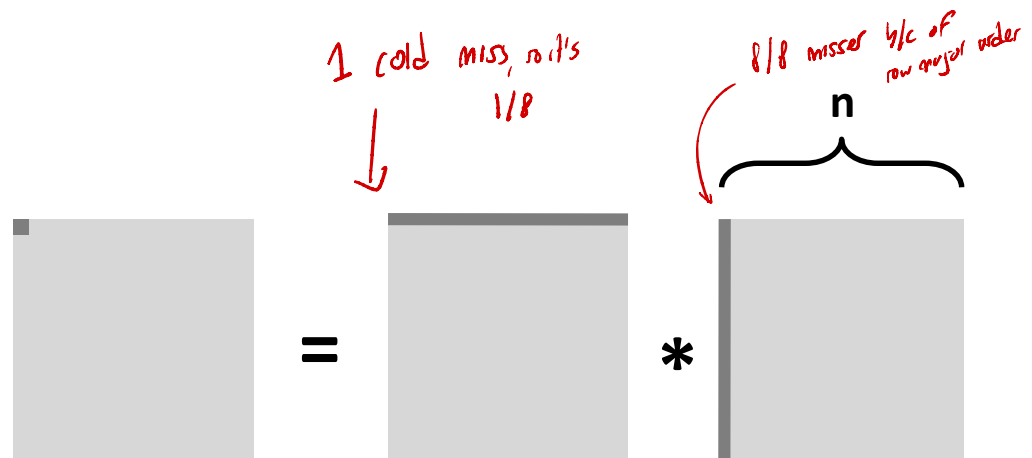
Cache Miss Analysis

■ Assume:

- Matrix elements are doubles, n is a multiple of 8
- Cache block = 8 doubles
- Cache size $S \ll n * \text{sizeof}(\text{double})$ (cache can't even hold one row/column)
- No allocate on write (no data in matrix c will be in cache)

■ How many total cache misses (not counting cache misses due to writes to matrix c)?

First iteration $i, j=0, k=0 \dots n-1$



$$\Rightarrow \frac{n}{8} + n = \frac{9n}{8}$$

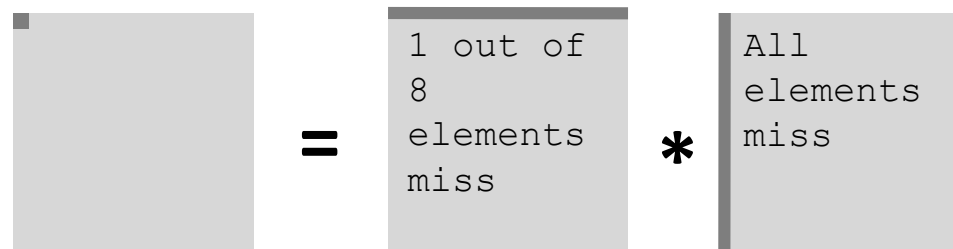
Cache Miss Analysis

■ Assume:

- Matrix elements are doubles, n is a multiple of 8
- Cache block = 8 doubles
- Cache size $S \ll n * \text{sizeof}(\text{double})$ (cache can't even hold one row/column)
- No allocate on write (no data in matrix c will be in cache)

■ First iteration $i, j=0, k=0 \dots n-1$: *the misses are the same, and we'll have n^2 iterations*

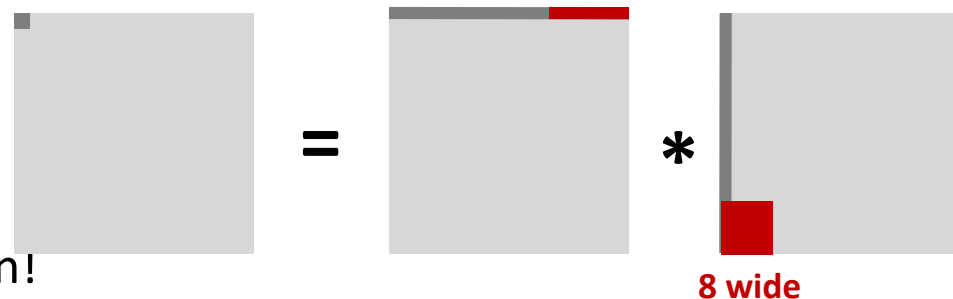
■ $n/8 + n = 9n/8$ misses



■ Red highlights the data that were last added to

The cache in this iteration

- But we don't need these data soon, i.e., at the beginning of the next iteration!



Cache Miss Analysis

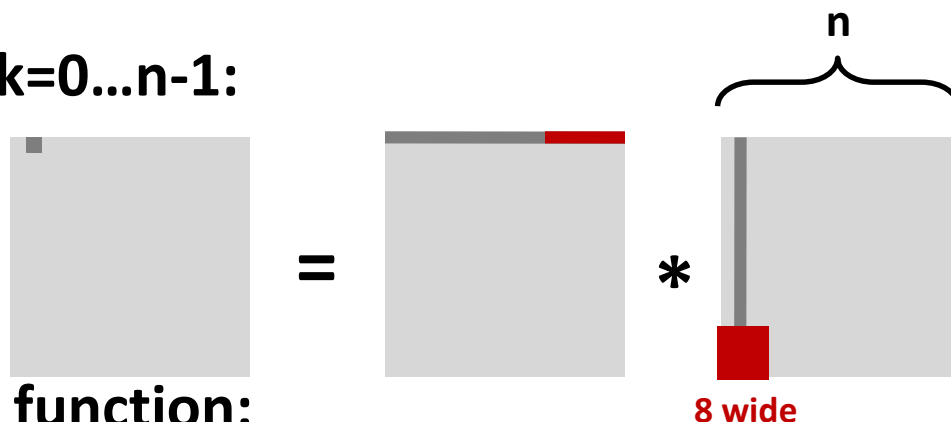
■ Assume:

- Matrix elements are doubles, n is a multiple of 8
- Cache block = 8 doubles
- Cache size $S \ll n * \text{sizeof}(\text{double})$ (cache can't even hold one row/column)
- No allocate on write (no data in matrix c will be in cache)

■ Second iteration, $i=0, j=1, k=0 \dots n-1$:

■ Again:

$$n/8 + n = 9n/8 \text{ misses}$$



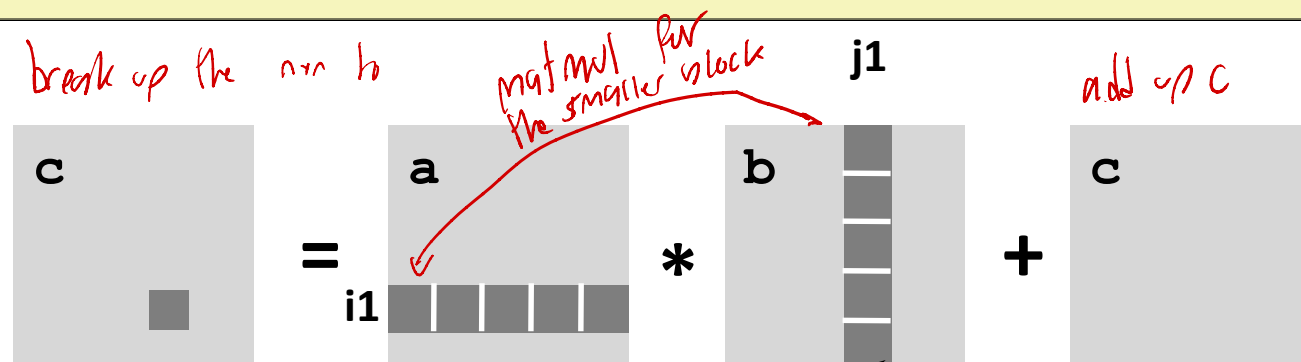
■ Total misses for the entire function:

- $9n/8 * n^2 = (9/8) * n^3$
 - Each inner loop: $9n/8$ misses
 - n^2 outer loops

Blocked Matrix Multiplication

```
c = (double *) calloc(sizeof(double), n*n);

/* Multiply n x n matrices a and b */
void mmm(double *a, double *b, double *c, int n) {
    int i, j, k;
    for (i = 0; i < n; i+=B)
        for (j = 0; j < n; j+=B)
            for (k = 0; k < n; k+=B)
                /* B x B mini matrix multiplications */
                for (i1 = i; i1 < i+B; i1++)
                    for (j1 = j; j1 < j+B; j1++)
                        for (k1 = k; k1 < k+B; k1++)
                            c[i1*n+j1] += a[i1*n + k1]*b[k1*n + j1];
}
```



Block size B doubles $\times$ B doubles

Note: B refers to the size of the matrix blocks (each block has $B \times B$ elements), not the block size of the cache

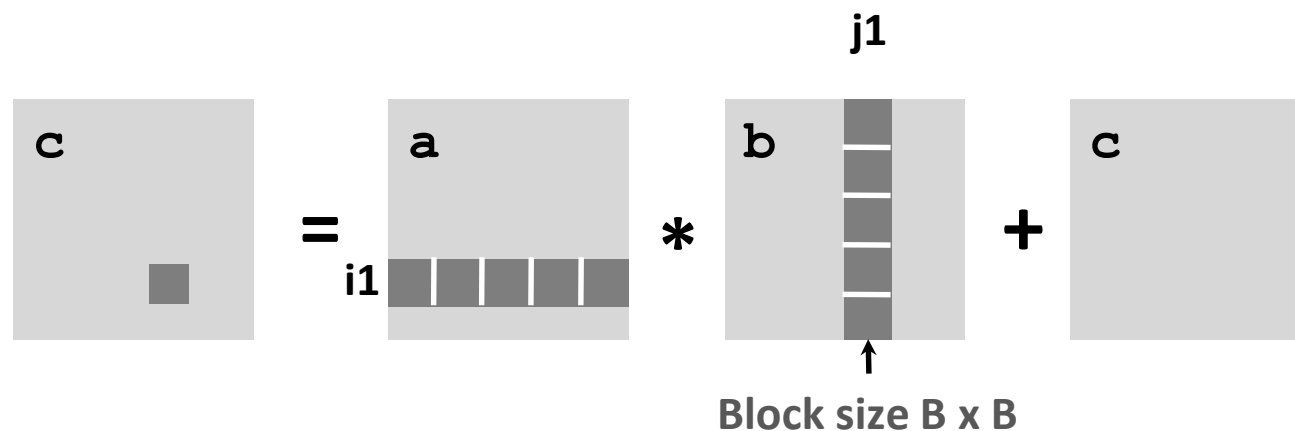
Blocked Matrix Multiplication

```
c = (double *) calloc(sizeof(double), n*n);

/* Multiply n x n matrices a and b */
void mmm(double *a, double *b, double *c, int n) {
    int i, j, k;
    for (i = 0; i < n; i += B)
        for (j = 0; j < n; j += B)
            for (k = 0; k < n; k += B)
                /* B x B mini matrix multiplications */
                for (i1 = i; i1 < i+B; i1++)
                    for (j1 = j; j1 < j+B; j1++)
                        for (k1 = k; k1 < k+B; k1++)
                            c[i1*n+j1] += a[i1*n+k1]*b[k1*n+j1];
}
```

Iterate through all small matrix blocks of a, b, and c

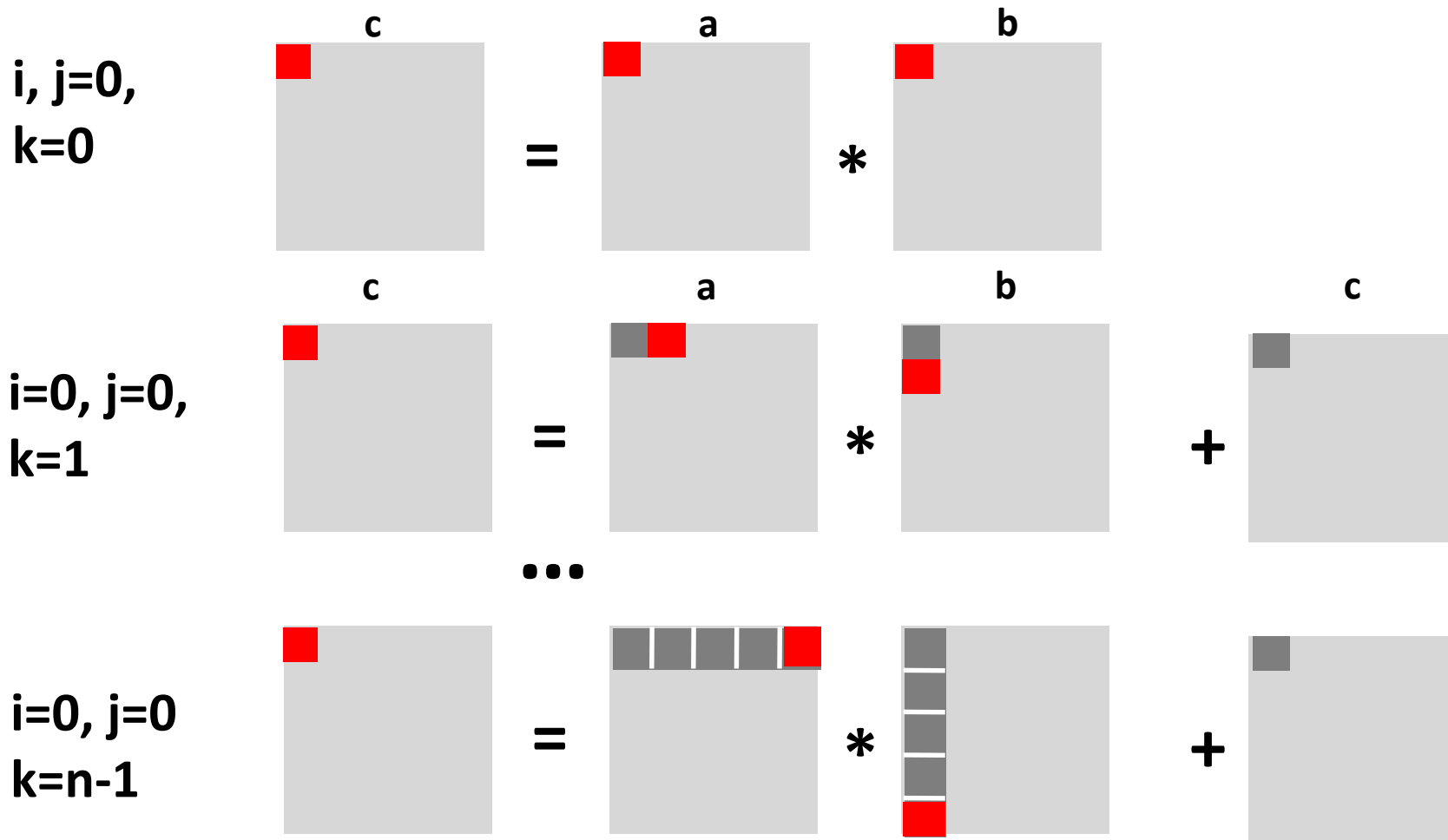
Matrix multiplication of two small matrix blocks



Note: B refers to the size of the matrix blocks (each block has BxB elements), not the block size of the cache

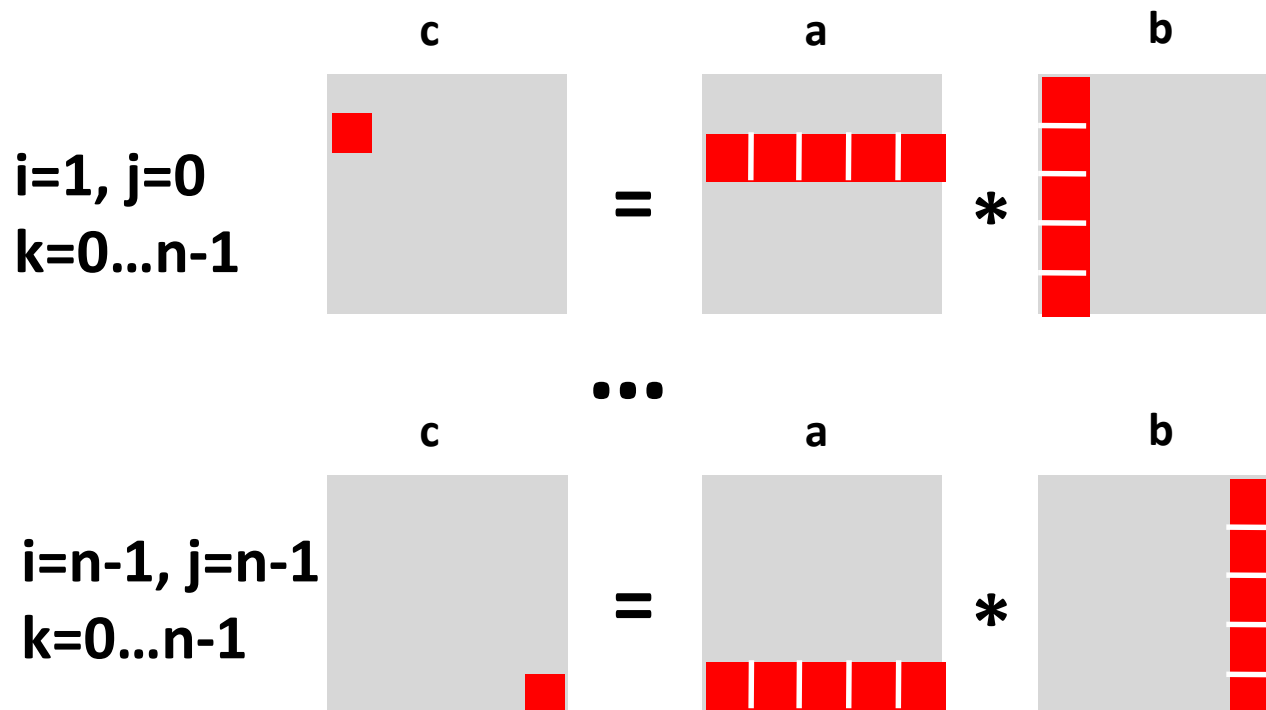
Blocked Matrix Multiplication

Blocking order:



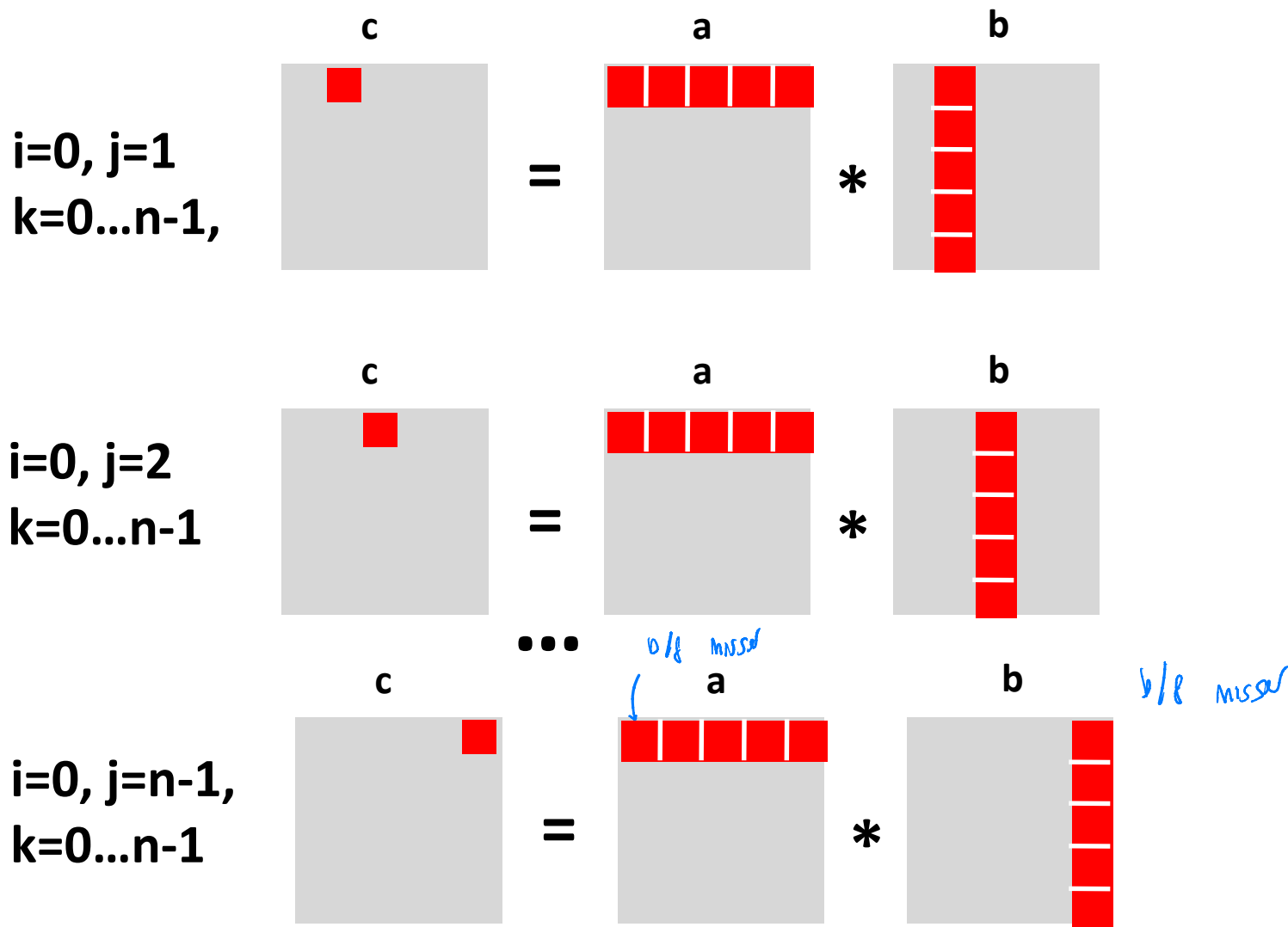
Blocked Matrix Multiplication

Blocking order:



Blocked Matrix Multiplication

Blocking order:



Cache Miss Analysis

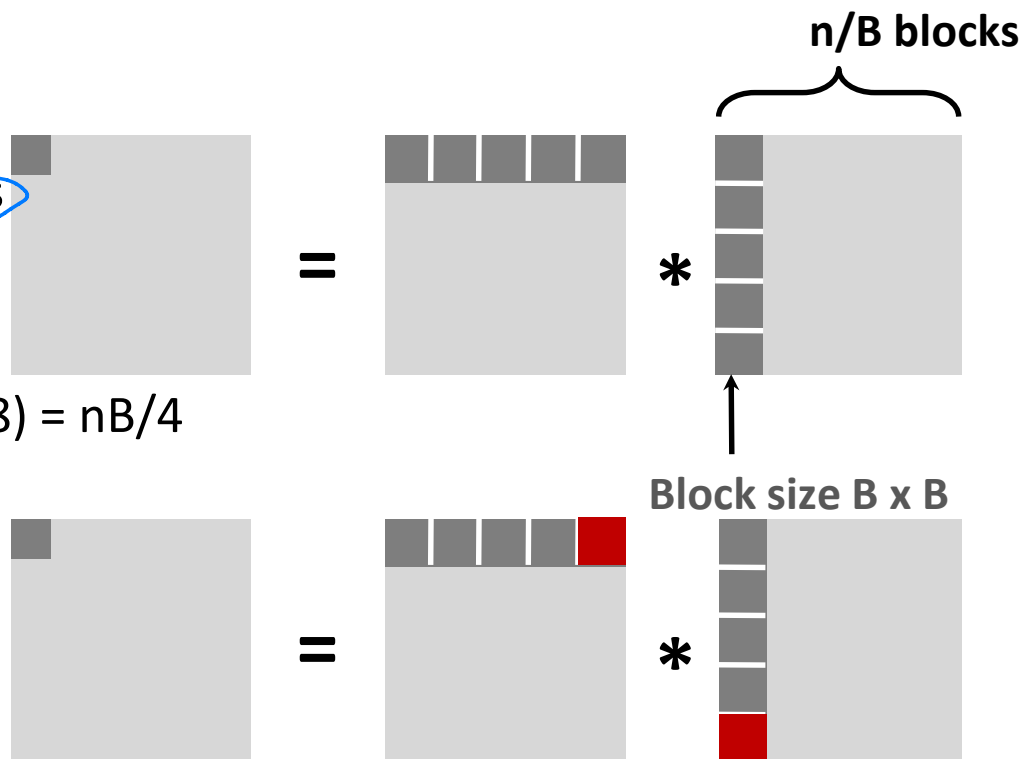
■ Assume:

- Cache block = 8 doubles
- Cache size $S \ll n * \text{sizeof}(\text{double})$ (cache can't even hold one row/column)
- Two blocks  fit into cache, B is a multiple of 8
- No allocate on write (no data in matrix c will be in cache)

■ First (block) iteration:

- $B^2/8$ misses for each block
 - 1 out of 8 elements miss
 - B^2 total elements
- $2n/B$ total blocks
- Total per iter.: $(2n/B) * (B^2/8) = nB/4$
- Red highlights data that are last added to the cache in this block iteration

1 iteration



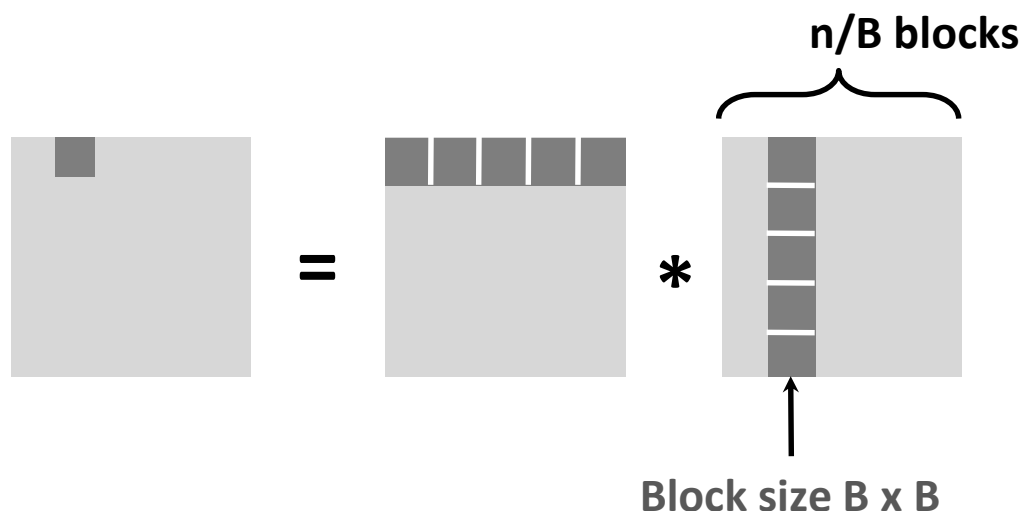
Cache Miss Analysis

■ Assume:

- Cache block = 8 doubles
- Cache size $S \ll n * \text{sizeof}(\text{double})$ (cache can't even hold one row/column)
- Two blocks  fit into cache, B is a multiple of 8
- No allocate on write (no data in matrix c will be in cache)

■ Every (block) iteration:

- Same as first iteration
- $2n/B * B^2/8 = nB/4$



■ Total misses:

- $nB/4 * (n/B)^2 = n^3/(4B)$
 - $nB/4$ misses per iteration
 - $(n/B)^2$ iterations

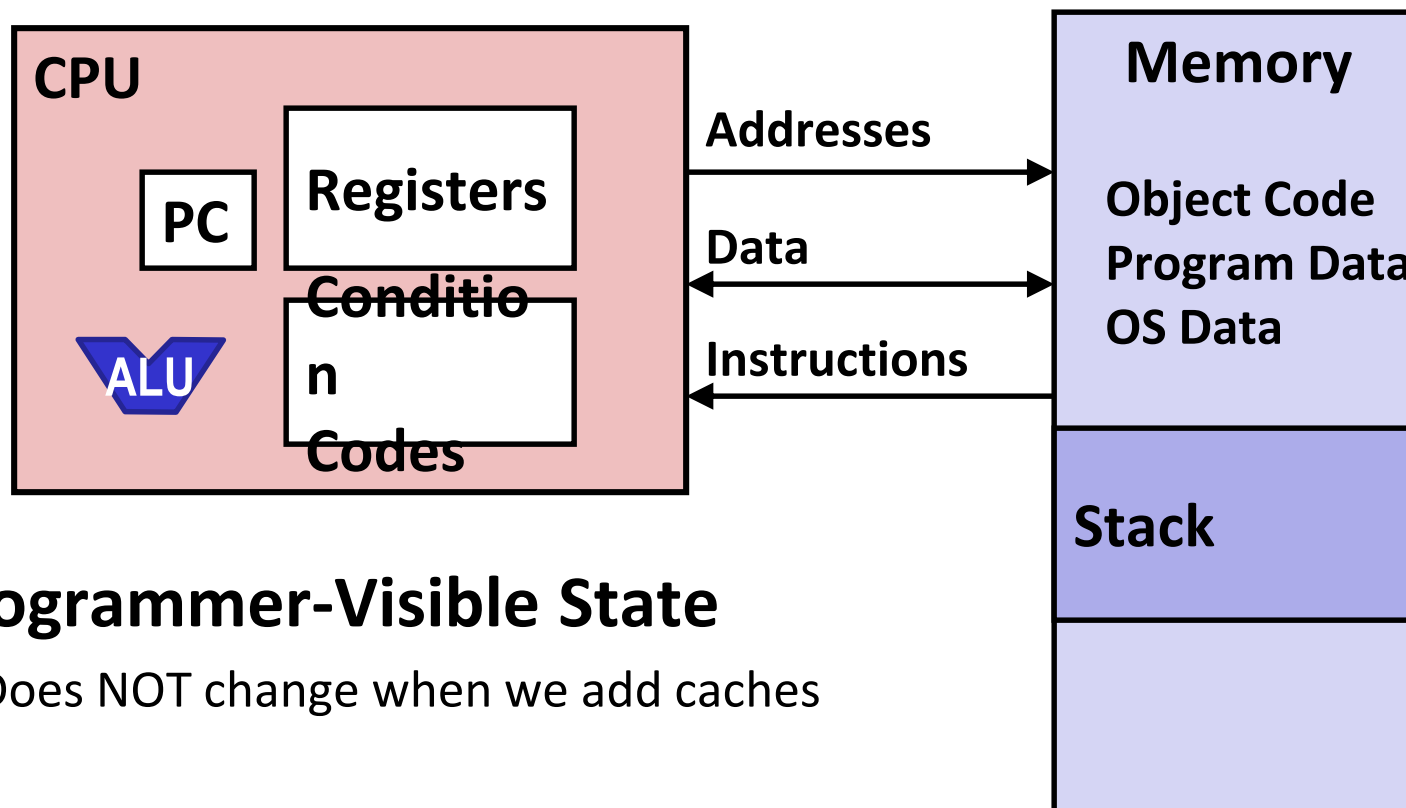
Blocking Summary

- No blocking: $(9/8) * n^3$
- Blocking: $1/(4B) * n^3$
- Suggest largest possible block size B s.t. $\text{sizeof(double)} * 2 * B^2 < S$
 - If $S = 64 \text{ KBytes} = 64 * 1024 \text{ Bytes}$, then setting $16 \text{ Bytes} * B^2 = S$ will give $B = 64$
 - $1/(4B) * n^3 \approx 0.004 n^3$, much smaller than $9/8 * n^3 \approx 1.1 n^3$
- Reason for dramatic difference:
 - Matrix multiplication has inherent temporal locality:
 - Total num. data: $2n^2$ (not counting matrix c), total num. multiplications: n^3
 - Every array element is used n times!
 - But program has to be written properly
 - Without blocking, we cannot leverage temporal locality at all
 - With blocking, data from cache can be used B times

Summary

- **Cache memories can have significant performance impact**
- **You can write your programs to improve performance!**
 - Focus on the inner loops, where bulk of computations and memory accesses occur.
 - Try to maximize spatial locality by reading data objects that are consecutive in memory.
 - Try to maximize temporal locality by using a data object as often as possible once it's read from memory
 - Optimize read over write

Software View of Architecture State



■ Programmer-Visible State

- Does NOT change when we add caches

■ Contrast with microarchitecture state

- Microarchitecture is the specific implementation of an ISA
 - HW designer decides what the microarchitecture looks like
- Caches: part of the microarchitecture state

Architecture vs. Microarchitecture

